🔐 CATEGORY 8: Security & Secret Management

> **Difficulty:** Intermediate → Advanced | **Topics:** 7 | **Terraform Interview Mastery Series**

Table of Contents

1. ⚠ Sensitive Variables and State File Exposure
2. Secrets in Terraform — Vault, SSM, Secrets Manager Integration
3. Least Privilege IAM for Terraform Execution Roles
4. OIDC-Based Auth for CI/CD — No Static Credentials
5. SAST Tools — `tfsec`, `checkov`, `terrascan`, `tflint`
6. ⚠ State Encryption — At Rest and In Transit
7. Supply Chain Security — Provider/Module Verification

Topic 55: ⚠ Sensitive Variables and State File Exposure

🔵 What It Is (Simple Terms)

Terraform's `sensitive = true` flag is widely misunderstood. It redacts values from CLI output — but does **nothing** to protect them in the state file, which stores every attribute in plaintext JSON. Understanding this gap is critical for secure Terraform usage.

🔵 What `sensitive = true` Actually Does

```
variable "db_password" {
  type      = string
  sensitive = true
}

resource "aws_db_instance" "main" {
  password = var.db_password
}

output "db_password" {
  value     = var.db_password
  sensitive = true
}
```

```
# What sensitive = true DOES:

# 1. Redacts from plan output
terraform plan
# aws_db_instance.main will be created
#   + password = (sensitive value)    ← redacted ✅

# 2. Redacts from apply output
# Apply complete! Resources: 1 added.

# 3. Redacts from terraform output
terraform output
# db_password = <sensitive>           ← redacted ✅

# 4. Errors if used in non-sensitive output (Terraform 0.15+)
output "connection_string" {
  value =
"postgres://:${var.db_password}@${aws_db_instance.main.endpoint}"
  # Error: Output "connection_string" includes sensitive values
  # without being marked as sensitive itself. Set sensitive = true
}
```

```
# What sensitive = true does NOT do:

# ❌ Does NOT encrypt state
cat .terraform/terraform.tfstate | grep password
# "password": "my-super-secret-password"  ← PLAINTEXT in state

# ❌ Does NOT prevent terraform output -raw from showing it
terraform output -raw db_password
# my-super-secret-password               ← SHOWN

# ❌ Does NOT prevent it from appearing in terraform output -json
terraform output -json
# { "db_password": { "sensitive": true, "value": "my-super-secret-
password" } }
# ← VALUE SHOWN IN JSON

# ❌ Does NOT protect it in TF_LOG=DEBUG logs
export TF_LOG=DEBUG
terraform apply 2>&1 | grep password
# may show password in HTTP request body
```

🔵 The Full Exposure Surface

```
WHERE SECRETS ARE EXPOSED:
```

```
1. State file (always — this is the biggest risk)
   Location: S3 bucket, local tfstate, TFC state storage
   Format: plaintext JSON
   Who can see it: anyone with read access to the state file
   Risk: database passwords, API keys, private keys, certificates

2. Plan output (mitigated by sensitive = true)
   Risk: CI/CD logs if sensitive = true not set

3. terraform output command (partially mitigated)
   -raw and -json flags bypass sensitive redaction

4. TF_LOG=DEBUG logs (not mitigated)
   HTTP request bodies may contain secrets

5. Provider logs
   Some providers log request/response bodies in debug mode

6. Remote execution logs (TFC/TFE)
   Run logs stored in Terraform Cloud — access controlled by workspace
permissions

7. Error messages
   Some provider errors include attribute values
   sensitive = true prevents this for known-sensitive attributes
```

🔵 Sensitive Value Propagation

```
# Sensitivity propagates through expressions automatically (TF 0.15+)

variable "db_password" {
  type      = string
  sensitive = true
}

locals {
  # This local is automatically sensitive
  # because it contains a sensitive value
  connection_string = "postgres://:${var.db_password}@mydb.example.com"
}

# Using a sensitive local in an output REQUIRES marking it sensitive
output "connection_string" {
  value     = local.connection_string
  sensitive = true   # REQUIRED — Terraform errors if you forget
}

# Using sensitive in a resource is fine — no special marking needed
resource "aws_ssm_parameter" "conn" {
  name  = "/app/db-connection"
```

```
  type  = "SecureString"
  value = local.connection_string   # works — sensitivity tracked
internally
}
```

🔵 What's Actually in State — The Full Exposure Reality

```
// Every single attribute stored in state — all plaintext

{
  "resources": [
    {
      "type": "aws_db_instance",
      "name": "production",
      "instances": [{
        "attributes": {
          "password":       "prod-db-p@ssw0rd-123!",
          "username":       "admin",
          "endpoint":       "prod-db.abc123.rds.amazonaws.com:5432",
          "address":        "prod-db.abc123.rds.amazonaws.com"
        }
      }]
    },
    {
      "type": "tls_private_key",
      "name": "server",
      "instances": [{
        "attributes": {
          "private_key_pem":      "-----BEGIN RSA PRIVATE KEY-----
\nMIIEo...",
          "private_key_openssh":  "-----BEGIN OPENSSH PRIVATE KEY-----
\nb3Bl...",
          "public_key_pem":       "-----BEGIN PUBLIC KEY-----\nMIIBI...",
          "public_key_openssh":   "ssh-rsa AAAAB3NzaC1..."
        }
      }]
    },
    {
      "type": "aws_secretsmanager_secret_version",
      "name": "api_key",
      "instances": [{
        "attributes": {
          "secret_string": "{\"api_key\": \"sk-prod-abc123xyz789\"}"
        }
      }]
    }
  ],
  "outputs": {
    "db_password": {
      "value":     "prod-db-p@ssw0rd-123!",  // ← PLAINTEXT even with
```

```
sensitive=true
      "sensitive": true
    }
  }
}
```

🔵 Mitigation Strategies

```
# Strategy 1: Don't put secrets in Terraform at all
# Use AWS RDS managed master password — Terraform never sees it
resource "aws_db_instance" "main" {
  identifier = "prod-db"
  engine     = "postgres"

  manage_master_user_password = true   # AWS generates + stores in Secrets
Manager
  # No password argument — Terraform never handles the credential
}

# Strategy 2: Generate + store, don't output
resource "random_password" "db" {
  length           = 32
  special          = true
  override_special = "!#$%&*()-_=+[]{}<>:?"
}

resource "aws_secretsmanager_secret_version" "db_password" {
  secret_id     = aws_secretsmanager_secret.db.id
  secret_string = random_password.db.result
  # random_password.db.result IS in state — but it's also safely in
Secrets Manager
  # Applications fetch from Secrets Manager, not from Terraform output
}

resource "aws_db_instance" "main" {
  password = random_password.db.result   # in state, but not exposed
externally
}

# Strategy 3: Fetch from external system at runtime
data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = aws_secretsmanager_secret.db_password.id
}
# This fetches the value at plan/apply time
# The secret_string IS stored in state (data source attributes are stored)
# But the value comes from Secrets Manager — not from Terraform input
```

🔵 Short Interview Answer

> "`sensitive = true` on a variable or output only redacts the value from CLI plan/apply output — it does NOT protect the value in the state file, which stores all attributes as plaintext JSON. This means the database password, private key, or API key you marked sensitive is fully readable by anyone with access to the state file. The real protection requires: S3 SSE-KMS encryption for the state file, strict IAM access control with separate roles per environment, and ideally minimizing secrets in Terraform entirely by using RDS managed passwords, fetching from Vault/Secrets Manager at runtime, or using services that generate and manage their own credentials."

🔵 Gotchas & Edge Cases

- ⚠ **`terraform output -json` shows sensitive values** — piping output to scripts or CI/CD can inadvertently log sensitive values if `-json` is used.
- ⚠ **Sensitive values in data sources** — data sources also store their attributes in state. `data.aws_secretsmanager_secret_version.db.secret_string` is in state in plaintext.
- **Sensitivity is module-scoped** — a sensitive value from a child module is automatically sensitive in the root module. You can't "un-sensitize" a value once marked.
- **Error messages may contain sensitive values** — even with `sensitive = true`, certain provider error messages may include attribute values in their text.

Topic 56: Secrets in Terraform — Vault, SSM, Secrets Manager Integration

---

🔵 What It Is (Simple Terms)

Best practice is to **not store secrets in Terraform at all** — but that's not always possible. This topic covers the patterns for integrating external secret stores (HashiCorp Vault, AWS SSM Parameter Store, AWS Secrets Manager) with Terraform to minimize secret exposure.

🔵 The Secret Handling Spectrum

```
WORST ←──────────────────────────────────────────────────────────→ BEST

Hardcoded       In tfvars     Generated &    Fetched from    Managed by
in .tf files    (committed)   stored in state external store  AWS directly
     ❌              ❌              ⚠              ✅              ✅✅

db_pass="abc"   db_pass="abc"   random_pwd     data.vault      manage_master_
                in prod.tfvars in state        .generic_sec
user_password=true
```

🔵 Pattern 1: AWS Secrets Manager Integration

```
# ── Fetch an existing secret (application created it) ─────────────────
data "aws_secretsmanager_secret" "db_password" {
  name = "/prod/rds/master-password"
}

data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = data.aws_secretsmanager_secret.db_password.id
}

resource "aws_db_instance" "main" {
  identifier     = "prod-db"
  engine         = "postgres"
  engine_version = "15.4"

  # Fetch from Secrets Manager — value stored in state but not in config
files
  password = jsondecode(
    data.aws_secretsmanager_secret_version.db_password.secret_string
  )["password"]
}

# ── Create and store a secret (Terraform manages it) ──────────────────
resource "random_password" "db" {
  length           = 32
  special          = true
  override_special = "!#$%&*()-_=+[]{}<>:?"
}

resource "aws_secretsmanager_secret" "db_password" {
  name                    = "/prod/rds/master-password"
  recovery_window_in_days = 7
  description             = "RDS master password for prod database"
}

resource "aws_secretsmanager_secret_version" "db_password" {
  secret_id = aws_secretsmanager_secret.db_password.id
  secret_string = jsonencode({
    password = random_password.db.result
    username = "admin"
    engine   = "postgres"
    host     = aws_db_instance.main.address
    port     = aws_db_instance.main.port
    dbname   = "appdb"
  })
}

# ── Best practice: AWS manages the password entirely ──────────────────
resource "aws_db_instance" "main" {
  identifier = "prod-db"
  engine     = "postgres"

  manage_master_user_password   = true           # AWS manages it
```

```
  master_user_secret_kms_key_id = aws_kms_key.rds.arn  # optional: CMK

  # Terraform NEVER sees the password
  # Access via: aws secretsmanager get-secret-value --secret-id <arn>
}

output "db_secret_arn" {
  # Give applications the ARN to fetch the secret themselves
  value = aws_db_instance.main.master_user_secret[0].secret_arn
}
```

🔵 Pattern 2: AWS SSM Parameter Store Integration

```
# ── Fetch existing SSM parameter ──────────────────────────────────
data "aws_ssm_parameter" "db_password" {
  name            = "/prod/app/db-password"
  with_decryption = true   # decrypt SecureString parameters
}

resource "aws_ecs_task_definition" "app" {
  family = "app"

  container_definitions = jsonencode([{
    name  = "app"
    image = var.docker_image

    secrets = [
      {
        name      = "DB_PASSWORD"
        valueFrom = data.aws_ssm_parameter.db_password.arn
        # ECS fetches from SSM at task start — not injected by Terraform
      }
    ]
  }])
}

# ── Store a secret in SSM ─────────────────────────────────────────
resource "aws_ssm_parameter" "api_key" {
  name        = "/prod/app/api-key"
  type        = "SecureString"          # encrypted with KMS
  value       = var.api_key             # var.api_key in state — use
TF_VAR_ to supply
  key_id      = aws_kms_key.ssm.arn     # customer-managed KMS key
  description = "Third-party API key for prod environment"

  tags = local.common_tags
}

# ── Read SSM parameters for cross-stack sharing ───────────────────
# Pattern: networking stack writes VPC info to SSM
```

```
resource "aws_ssm_parameter" "vpc_id" {
  name  = "/prod/networking/vpc-id"
  type  = "String"
  value = aws_vpc.main.id
}

# Application stack reads — no state dependency
data "aws_ssm_parameter" "vpc_id" {
  name = "/prod/networking/vpc-id"
}
```

🔵 Pattern 3: HashiCorp Vault Integration

```
# ── Vault Provider Configuration ──────────────────────────────────
provider "vault" {
  address = "https://vault.mycompany.com:8200"

  # Authentication: use approle or AWS auth — not token in config
  # In CI/CD: VAULT_TOKEN env var or VAULT_ROLE_ID + VAULT_SECRET_ID
}

# ── Fetch a secret from Vault KV v2 ───────────────────────────────
data "vault_kv_secret_v2" "db_password" {
  mount = "secret"
  name  = "prod/rds/master"
}

resource "aws_db_instance" "main" {
  password = data.vault_kv_secret_v2.db_password.data["password"]
  # ⚠ This value IS stored in Terraform state
  # Vault is the authoritative store, but state has a copy
}

# ── Vault dynamic secrets — the gold standard ─────────────────────
# Vault generates time-limited credentials — no static secret to store

data "vault_aws_access_credentials" "deploy" {
  backend = "aws"
  role    = "deploy-role"
  # Vault generates temporary AWS access key + secret for this role
  # Expires automatically — no long-lived credential
}

provider "aws" {
  access_key = data.vault_aws_access_credentials.deploy.access_key
  secret_key = data.vault_aws_access_credentials.deploy.secret_key
}

# ── Vault PKI — dynamic TLS certificates ──────────────────────────
data "vault_pki_secret_backend_cert" "app" {
```

```
  backend     = vault_mount.pki.path
  name        = vault_pki_secret_backend_role.server.name
  common_name = "app.internal.mycompany.com"
  ttl         = "720h"   # 30-day cert, Vault regenerates at renewal
}

resource "aws_acm_certificate" "app" {
  private_key       = data.vault_pki_secret_backend_cert.app.private_key
  certificate_body  = data.vault_pki_secret_backend_cert.app.certificate
  certificate_chain = data.vault_pki_secret_backend_cert.app.ca_chain
}
```

🔵 Pattern 4: Environment Variables for Secret Injection

```
# The safest Terraform input path — never written to disk

# CI/CD injects secrets as environment variables
export TF_VAR_db_password="${DB_PASSWORD}"      # from CI secret store
export TF_VAR_api_key="${API_KEY}"
export TF_VAR_vault_token="${VAULT_TOKEN}"

terraform apply
# Terraform reads TF_VAR_db_password → uses as variable value
# Never written to any file — only in memory during apply
# ⚠ Still ends up in state if used in a resource attribute
```

🔵 The Hierarchy of Least Exposure

```
Level 1: AWS manages the secret entirely
  manage_master_user_password = true
  → Secret never seen by Terraform at all ✅✅

Level 2: Secret generated and stored — not input by user
  random_password → aws_secretsmanager_secret_version
  → In state, but sourced securely ✅

Level 3: Secret fetched from external store at apply time
  data.vault_kv_secret_v2 or data.aws_ssm_parameter
  → In state (data sources store attributes) ⚠

Level 4: Secret injected via TF_VAR_ env variable
  Never in files, only in memory during apply
  → In state if used in resource ⚠

Level 5: Secret in tfvars file (committed)
  → In git history, in state ❌
```

```
Level 6: Secret hardcoded in .tf file
  → In git history, in state, visible to all ❌❌
```

🔵 Short Interview Answer

> "The best approach to secrets in Terraform is not handling them in Terraform at all — use RDS `manage_master_user_password = true` so AWS generates and stores the credential. When Terraform must handle secrets, prefer fetching from Vault or Secrets Manager via data sources rather than injecting as variables — though note that data source attribute values ARE stored in state. For CI/CD secret injection, `TF_VAR_` environment variables are better than tfvars files because they're never written to disk. The universal rule: state file encryption is non-negotiable because every secret handling approach eventually results in values appearing in state."

Topic 57: Least Privilege IAM for Terraform Execution Roles

🔵 What It Is (Simple Terms)

The IAM role that Terraform uses to make AWS API calls should have **exactly the permissions needed to manage the resources in that config — nothing more**. This limits the blast radius if credentials are compromised or if Terraform is misconfigured.

🔵 Why Least Privilege Matters for Terraform

```
Without least privilege:
  Terraform role has AdministratorAccess
  → Terraform bug creates 1000 EC2 instances by accident
  → Compromised Terraform token = full AWS account control
  → Developer gets access to Terraform role = all prod access
  → Misuse: developer uses Terraform role for other purposes

With least privilege:
  Terraform role has only permissions for VPC + EC2 in us-east-1
  → Accidental IAM policy creation = permission denied
  → Compromised token can only affect managed resources
  → Role is scoped to environment and component
```

🔵 Designing Least Privilege Terraform Roles

```
Principle: One IAM role per state file (per Terraform workspace)

Role structure:
  TerraformRole-Prod-Networking   → manages VPC, subnets, routes
  TerraformRole-Prod-Security     → manages IAM, security groups, KMS
  TerraformRole-Prod-Compute      → manages EC2, EKS, ASG, Lambda
  TerraformRole-Prod-Data         → manages RDS, ElastiCache, S3
  TerraformRole-Dev-All           → all permissions in dev (lower risk)

Benefits:
  → Compromise of one role affects only one component
  → Access reviews are component-scoped
  → Easier to audit "what can Terraform change in prod networking?"
```

🔵 Discovering Required Permissions

```
# Method 1: AWS IAM Access Analyzer — run with broad permissions,
# check what was actually called

# Enable IAM Access Analyzer
aws accessanalyzer create-analyzer \
  --analyzer-name terraform-permission-analyzer \
  --type ACCOUNT

# Run Terraform apply with CloudTrail enabled
# Then generate least-privilege policy from CloudTrail:
aws iam generate-service-last-accessed-details \
  --arn arn:aws:iam::123456789012:role/TerraformRole-Temp

# Method 2: iamlive — intercepts API calls and generates policy
pip install iamlive
# export AWS_CA_BUNDLE=~/.iamlive/ca.pem
# export HTTP_PROXY=http://127.0.0.1:10080
# export HTTPS_PROXY=http://127.0.0.1:10080
iamlive &
terraform apply
iamlive stop
# → Prints minimum required IAM policy

# Method 3: tfsec / checkov — validates resource-level permissions in
config
# (see Topic 59)
```

🔵 IAM Policy Structure for Terraform Roles

```
// TerraformRole-Prod-Networking Policy

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VPCManagement",
      "Effect": "Allow",
      "Action": [
        "ec2:CreateVpc", "ec2:DeleteVpc", "ec2:DescribeVpcs",
        "ec2:ModifyVpcAttribute", "ec2:AssociateVpcCidrBlock",
        "ec2:CreateSubnet", "ec2:DeleteSubnet", "ec2:DescribeSubnets",
        "ec2:ModifySubnetAttribute",
        "ec2:CreateRouteTable", "ec2:DeleteRouteTable",
        "ec2:AssociateRouteTable", "ec2:DisassociateRouteTable",
        "ec2:CreateRoute", "ec2:DeleteRoute", "ec2:DescribeRouteTables",
        "ec2:CreateInternetGateway", "ec2:DeleteInternetGateway",
        "ec2:AttachInternetGateway", "ec2:DetachInternetGateway",
        "ec2:DescribeInternetGateways",
        "ec2:AllocateAddress", "ec2:ReleaseAddress",
"ec2:DescribeAddresses",
        "ec2:CreateNatGateway", "ec2:DeleteNatGateway",
"ec2:DescribeNatGateways",
        "ec2:CreateTags", "ec2:DeleteTags", "ec2:DescribeTags",
        "ec2:DescribeAvailabilityZones"
      ],
      "Resource": "*"
    },
    {
      "Sid": "StateAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject", "s3:PutObject", "s3:DeleteObject", "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::mycompany-terraform-state",
        "arn:aws:s3:::mycompany-terraform-state/prod/networking/*"
      ]
    },
    {
      "Sid": "StateLocking",
      "Effect": "Allow",
      "Action": ["dynamodb:GetItem", "dynamodb:PutItem",
"dynamodb:DeleteItem"],
      "Resource": "arn:aws:dynamodb:us-east-
1:123456789012:table/terraform-state-locks"
    },
    {
      "Sid": "StateEncryption",
      "Effect": "Allow",
      "Action": ["kms:GenerateDataKey", "kms:Decrypt", "kms:DescribeKey"],
      "Resource": "arn:aws:kms:us-east-1:123456789012:key/abc123"
    }
```

```
  ]
}
```

🔵 Trust Policy — Who Can Assume the Role

```
// Trust policy for CI/CD using OIDC (GitHub Actions)
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::123456789012:oidc-
provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          // Only allow from specific repo and branch
          "token.actions.githubusercontent.com:sub":
            "repo:myorg/infra-repo:ref:refs/heads/main"
        }
      }
    }
  ]
}
```

🔵 Permission Boundaries — Additional Guardrail

```
# Permission boundaries limit what a role can do even if the policy is too
broad
# Terraform role can never exceed the boundary

resource "aws_iam_role" "terraform_networking" {
  name = "TerraformRole-Prod-Networking"

  assume_role_policy   = data.aws_iam_policy_document.github_oidc.json
  permissions_boundary = aws_iam_policy.terraform_boundary.arn

  # Even if someone accidentally attaches AdministratorAccess to this
role,
  # the permission boundary limits actual effective permissions
}
```

```
resource "aws_iam_policy" "terraform_boundary" {
  name = "TerraformExecutionBoundary"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect   = "Allow"
        Action   = ["ec2:*", "elasticloadbalancing:*", "autoscaling:*"]
        Resource = "*"
        Condition = {
          StringEquals = { "aws:RequestedRegion" = "eu-west-1" }  # region
lock
        }
      },
      {
        Effect   = "Deny"
        Action   = ["iam:*", "organizations:*", "account:*"]   # never
allow IAM changes
        Resource = "*"
      }
    ]
  })
}
```

🔵 Short Interview Answer

> "Least privilege for Terraform means one IAM role per state file, scoped to the exact AWS actions needed to manage those specific resources. Design it by running Terraform against CloudTrail with IAM Access Analyzer, or using tools like `iamlive` to capture actual API calls. The role policy includes permissions for the managed resources plus the state backend (S3 GetObject/PutObject, DynamoDB PutItem/DeleteItem, KMS Decrypt). The trust policy restricts which principals can assume the role — for GitHub Actions OIDC, constrain to a specific repo and branch. Add permission boundaries as a defense-in-depth guardrail that caps permissions even if the policy is misconfigured."

Topic 58: OIDC-Based Auth for CI/CD — No Static Credentials

🔵 What It Is (Simple Terms)

OpenID Connect (OIDC) allows CI/CD systems like GitHub Actions and GitLab CI to **authenticate to AWS without storing static access keys**. The CI/CD platform issues a short-lived JWT token; AWS verifies it and exchanges it for temporary credentials via `sts:AssumeRoleWithWebIdentity`.

🔵 The Problem with Static Credentials

```
Traditional approach:
  1. Create IAM user with access keys
  2. Store access_key_id + secret_access_key in GitHub Secrets
  3. CI/CD job uses them to run terraform apply

Problems:
  ❌ Long-lived credentials — one leak = persistent compromise
  ❌ Key rotation is manual and disruptive
  ❌ Keys stored in many places — GitHub, developer machines, CI logs
  ❌ Hard to audit which systems use which keys
  ❌ No automatic expiry — stolen keys work until manually rotated
  ❌ Violates principle of least privilege for time

OIDC approach:
  ✅ No credentials stored anywhere — GitHub holds a private signing key
  ✅ Short-lived tokens (15 min - 1 hour) — compromise window is tiny
  ✅ Automatic — no rotation needed
  ✅ Auditable — every assumption shows in CloudTrail with full context
  ✅ Granular — restrict to specific repo, branch, environment
```

🔵 How OIDC Works — The Flow

```
1. GitHub Actions job starts
2. GitHub mints a JWT (OIDC token) signed with GitHub's private key
   JWT claims include:
   {
     "sub": "repo:myorg/infra-repo:ref:refs/heads/main",
     "aud": "sts.amazonaws.com",
     "iss": "https://token.actions.githubusercontent.com",
     "ref": "refs/heads/main",
     "repository": "myorg/infra-repo",
     "workflow": "Deploy"
   }

3. GitHub Actions calls AWS STS:
   sts:AssumeRoleWithWebIdentity(
     RoleArn = "arn:aws:iam::123456789012:role/TerraformRole-Prod",
     WebIdentityToken = <JWT>,
     RoleSessionName = "github-deploy"
   )

4. AWS verifies the JWT against GitHub's public keys
   (fetched from https://token.actions.githubusercontent.com/.well-
known/jwks)

5. AWS checks trust policy conditions match the JWT claims
```

```
6. AWS returns temporary credentials:
   AccessKeyId:     ASIA...
   SecretAccessKey: wJalr...
   SessionToken:    IQoJb... (expires in 1 hour)

7. GitHub Actions uses these temporary credentials for terraform apply

8. After 1 hour: credentials expire automatically — no rotation needed
```

🔵 Setting Up OIDC — AWS Side

```
# Step 1: Create the OIDC Identity Provider in AWS
resource "aws_iam_openid_connect_provider" "github" {
  url = "https://token.actions.githubusercontent.com"

  client_id_list = ["sts.amazonaws.com"]

  # GitHub's thumbprint — get from:
  # openssl s_client -servername token.actions.githubusercontent.com \
  #   -connect token.actions.githubusercontent.com:443 < /dev/null
2>/dev/null \
  #   | openssl x509 -fingerprint -sha1 -noout
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

# Step 2: Create IAM role with trust policy
resource "aws_iam_role" "terraform_deploy" {
  name = "TerraformRole-Prod-Deploy"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Principal = {
          Federated = aws_iam_openid_connect_provider.github.arn
        }
        Action = "sts:AssumeRoleWithWebIdentity"
        Condition = {
          StringEquals = {
            "token.actions.githubusercontent.com:aud" =
"sts.amazonaws.com"
          }
          StringLike = {
            # Only allow from specific org/repo on main branch
            "token.actions.githubusercontent.com:sub" = [
              "repo:myorg/infra-repo:ref:refs/heads/main",
              "repo:myorg/infra-repo:environment:production"
            ]
          }
```

```
        }
      }
    ]
  })

  # Max session duration — 1 hour is usually sufficient
  max_session_duration = 3600
}

# Step 3: Attach least-privilege policy
resource "aws_iam_role_policy_attachment" "terraform_deploy" {
  role       = aws_iam_role.terraform_deploy.name
  policy_arn = aws_iam_policy.terraform_networking.arn
}
```

🔵 GitHub Actions Workflow

```yaml
# .github/workflows/terraform-apply.yml

name: Terraform Apply

on:
  push:
    branches: [main]

permissions:
  id-token: write    # REQUIRED: allows the workflow to request OIDC token
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production    # maps to GitHub environment for additional
controls

    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/TerraformRole-
Prod-Deploy
          aws-region: eu-west-1
          # No access_key_id or secret_access_key — pure OIDC!
          role-session-name: github-terraform-${{ github.run_id }}

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
```

```
          terraform_version: "1.6.0"

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        id: plan
        run: terraform plan -out=tfplan -detailed-exitcode
        continue-on-error: true

      - name: Terraform Apply
        if: steps.plan.outputs.exitcode == '2'   # only apply if there are
changes
        run: terraform apply -auto-approve tfplan
```

🔵 GitLab CI OIDC

```
# .gitlab-ci.yml
deploy:
  image: hashicorp/terraform:1.6.0
  id_tokens:
    AWS_OIDC_TOKEN:
      aud: sts.amazonaws.com   # audience must match trust policy

  script:
    - |
      # Exchange GitLab OIDC token for AWS credentials
      CREDENTIALS=$(aws sts assume-role-with-web-identity \
        --role-arn "arn:aws:iam::123456789012:role/TerraformRole-Prod" \
        --role-session-name "gitlab-deploy-${CI_JOB_ID}" \
        --web-identity-token "${AWS_OIDC_TOKEN}" \
        --duration-seconds 3600 \
        --query "Credentials" \
        --output json)

      export AWS_ACCESS_KEY_ID=$(echo $CREDENTIALS | jq -r '.AccessKeyId')
      export AWS_SECRET_ACCESS_KEY=$(echo $CREDENTIALS | jq -r
'.SecretAccessKey')
      export AWS_SESSION_TOKEN=$(echo $CREDENTIALS | jq -r
'.SessionToken')

      terraform init
      terraform apply -auto-approve
```

🔵 Trust Policy Conditions — Granular Control

```
// Different conditions you can enforce in the trust policy:

// 1. Specific repository only
"repo:myorg/infra-repo:*"

// 2. Specific branch only (production deployments from main only)
"repo:myorg/infra-repo:ref:refs/heads/main"

// 3. Specific GitHub Environment (with required reviewers)
"repo:myorg/infra-repo:environment:production"

// 4. Any branch in the repo (for plan-only role)
"repo:myorg/infra-repo:pull_request"

// Multiple repos (shared infrastructure team)
"repo:myorg/infra-repo:*",
"repo:myorg/app-repo:environment:production"

// 5. Specific workflow
{
  "StringLike": {
    "token.actions.githubusercontent.com:sub": "repo:myorg/infra-repo:*"
  },
  "StringEquals": {
    "token.actions.githubusercontent.com:workflow": "Terraform Apply"
  }
}
```

🔵 Plan vs Apply Role Separation

```
# Best practice: separate roles for plan (read) and apply (write)

# Plan role — read-only + state read
resource "aws_iam_role" "terraform_plan" {
  name = "TerraformRole-Prod-Plan"
  # Trust: any branch (PRs trigger plans)
  # Permissions: read-only AWS APIs + S3 GetObject (state read)
}

# Apply role — full permissions + state write
resource "aws_iam_role" "terraform_apply" {
  name = "TerraformRole-Prod-Apply"
  # Trust: main branch only
  # Permissions: full resource management + S3 PutObject (state write)
}
```

```
# GitHub Actions: plan on PR, apply on merge to main
on:
  pull_request:
    # runs plan with read-only role
  push:
    branches: [main]
    # runs apply with write role
```

🔵 Short Interview Answer

> "OIDC authentication eliminates static credentials entirely. GitHub Actions (or GitLab CI) issues a short-lived JWT token signed by the CI platform. Terraform's AWS provider exchanges this token for temporary AWS credentials via `sts:AssumeRoleWithWebIdentity`. The AWS trust policy validates the token claims — you can restrict to a specific repository, branch, and GitHub Environment. Setup requires: creating an IAM OIDC provider in AWS pointing to the CI platform's discovery URL, creating an IAM role with the trust policy specifying claim conditions, and granting `id-token: write` permission in the GitHub workflow. No credentials are stored anywhere — the CI platform's private signing key is the only secret, and credentials expire automatically after the session."

Topic 59: SAST Tools — `tfsec`, `checkov`, `terrascan`, `tflint`

🔵 What It Is (Simple Terms)

Static Application Security Testing (SAST) for Terraform means analyzing your `.tf` files for security misconfigurations, policy violations, and code quality issues — **before** `terraform apply` runs. Catch problems at the PR stage, not in production.

🔵 The Four Tools — What Each Does

```
┌──────────────────────────────────────────────────────────────┐
│  Tool          Primary Focus           Language   Best For    │
├──────────────────────────────────────────────────────────────┤
│  tfsec         Security misconfigs     Go         Security scanning │
│                (S3 public access,                  Fast, low setup   │
│                SGs open to world,                                    │
│                unencrypted resources)                                │
│                                                                      │
│  checkov       Security + compliance   Python     Multi-framework   │
│                (CIS benchmarks,                    (TF, CF, K8s,     │
│                SOC2, PCI-DSS,                       ARM, Helm)       │
```

```
│                HIPAA checks)                                          │
│                                                                       │
│  terrascan     Policy as code          Go         OPA/Rego policies  │
│                (custom Rego rules,                 Enterprise policy │
│                compliance frameworks)              enforcement        │
│                                                                       │
│  tflint        Code quality +          Go         Linting, best      │
│                provider rules                      practices,         │
│                (invalid instance types,            provider-specific │
│                deprecated args,                    validation         │
│                naming conventions)                                    │
└──────────────────────────────────────────────────────────────────────┘
```

🔵 `tfsec` — Security Scanner

```
# Install
brew install tfsec
# or: curl -s
https://raw.githubusercontent.com/aquasecurity/tfsec/master/scripts/instal
l_linux.sh | bash

# Basic scan
tfsec .

# Scan with specific severity threshold
tfsec . --minimum-severity HIGH

# Output formats
tfsec . --format json > tfsec-results.json
tfsec . --format sarif > tfsec-results.sarif  # GitHub Security tab
tfsec . --format junit > tfsec-results.xml    # JUnit for CI/CD

# Example findings:
# [HIGH] aws-s3-enable-bucket-encryption - S3 bucket not encrypted
#   /modules/storage/main.tf:5:1
#   resource "aws_s3_bucket" "data" {
#
# [HIGH] aws-ec2-no-public-ingress-sgr - Security group allows ingress
from 0.0.0.0/0 on port 22
#   /modules/compute/security_groups.tf:15:3
#
# [MEDIUM] aws-rds-enable-deletion-protection - RDS instance has deletion
protection disabled
```

```
# Suppress a specific check with a comment annotation
resource "aws_s3_bucket" "public_assets" {
  #tfsec:ignore:aws-s3-block-public-acls
  #tfsec:ignore:aws-s3-block-public-policy
```

```
  bucket = "myapp-public-assets"
  # ↑ Intentionally public — serves static assets
}
```

🔵 checkov — Compliance Scanner

```
# Install
pip install checkov

# Basic scan
checkov -d .

# Scan specific file
checkov -f main.tf

# Filter by framework
checkov -d . --framework terraform

# Filter by specific checks
checkov -d . --check CKV_AWS_18,CKV_AWS_20

# Skip specific checks
checkov -d . --skip-check CKV_AWS_18

# Output formats
checkov -d . -o json
checkov -d . -o junitxml > checkov-results.xml
checkov -d . -o sarif > checkov-results.sarif

# Example output:
# Check: CKV_AWS_18: "Ensure the S3 bucket has access logging enabled"
#   FAILED for resource: aws_s3_bucket.data
#   File: /modules/storage/main.tf:1-10
#
# Check: CKV_AWS_20: "Ensure the S3 bucket has MFA delete enabled"
#   FAILED for resource: aws_s3_bucket.state
#   File: /bootstrap/main.tf:5-15
```

```
# Suppress a check inline
resource "aws_security_group" "bastion" {
  #checkov:skip=CKV_AWS_24:Bastion host needs SSH access from corporate
IPs
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["203.0.113.0/24"]  # corporate IP range
```

```
  }
}
```

🔵 `tflint` — Linter and Best Practices

```
# Install
brew install tflint
# or: curl -s https://raw.githubusercontent.com/terraform-
linters/tflint/master/install_linux.sh | bash

# Initialize with provider plugins
tflint --init

# Basic lint
tflint

# With specific ruleset
tflint --enable-rule=terraform_documented_variables
tflint --enable-rule=terraform_naming_convention

# Example .tflint.hcl configuration
cat > .tflint.hcl << 'EOF'
plugin "aws" {
  enabled = true
  version = "0.27.0"
  source  = "github.com/terraform-linters/tflint-ruleset-aws"
}

rule "terraform_documented_variables" {
  enabled = true
}

rule "terraform_naming_convention" {
  enabled = true
  format  = "snake_case"
}

rule "aws_instance_invalid_type" {
  enabled = true
}
EOF

tflint --config .tflint.hcl
```

```
# What tflint catches:

# 1. Invalid resource types (not just linting — provider validation)
# Error: "t2.micr" is an invalid value as instance_type
```

```
# ← Catches typos before apply!

# 2. Deprecated arguments
# Warning: "associate_public_ip_address" was deprecated in AWS provider
v4.0

# 3. Missing documentation
# Warning: variable "environment" is missing description

# 4. Naming conventions
# Warning: Resource name "WebServer" should be snake_case: "web_server"

# 5. Required tags policy (with custom rules)
# Error: Resource aws_instance.web is missing required tag "Environment"
```

🔵 `terrascan` — Policy as Code

```
# Install
brew install terrascan
# or: curl -L
"https://github.com/tenable/terrascan/releases/latest/download/terrascan_L
inux_x86_64.tar.gz" | tar -xz

# Basic scan
terrascan scan -t aws

# Scan with output format
terrascan scan -t aws -o json
terrascan scan -t aws -o junit-xml

# Custom Rego policy example
cat > policies/require-tags.rego << 'EOF'
package accurics.terraform.IAM.requiredTags

import future.keywords

deny[msg] {
  resource := input.resource.aws_instance[_]
  not resource.config.tags.Environment
  msg := sprintf("Resource %v is missing required tag 'Environment'",
[resource.name])
}
EOF

terrascan scan -t aws --policy-path ./policies/
```

🔵 CI/CD Integration — The Full Pipeline

```yaml
# .github/workflows/terraform-security.yml

name: Terraform Security Scan

on:
  pull_request:
    paths: ['**.tf', '**.tfvars']

jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # tflint — syntax and provider validation
      - uses: terraform-linters/setup-tflint@v4
      - name: tflint
        run: |
          tflint --init
          tflint --format=compact
        continue-on-error: false   # fail on lint errors

      # tfsec — security scanning
      - name: tfsec
        uses: aquasecurity/tfsec-action@v1.0.0
        with:
          minimum_severity: HIGH
          soft_fail: false         # fail PR on HIGH findings

      # checkov — compliance scanning
      - name: checkov
        uses: bridgecrewio/checkov-action@master
        with:
          directory: .
          framework: terraform
          output_format: sarif
          output_file_path: reports/results.sarif
          soft_fail: false

      # Upload results to GitHub Security tab
      - name: Upload SARIF
        uses: github/codeql-action/upload-sarif@v2
        if: always()
        with:
          sarif_file: reports/results.sarif
```

🔵 Comparison: Which Tool to Use When

```
tflint first:
  Fast, catches syntax/naming before security tools run
  Provider-specific validation (invalid instance types, deprecated args)
  Use on every plan/PR

tfsec second:
  Security-focused, fast, Terraform-native
  Best for AWS/GCP/Azure security misconfiguration scanning
  Good default choice for most teams

checkov if you need:
  Multi-framework scanning (you also use CloudFormation, Helm, Kubernetes)
  Specific compliance frameworks (CIS, NIST, SOC2)
  More comprehensive check library than tfsec

terrascan if you need:
  Custom Rego policies (organization-specific rules)
  Complex policy logic that check/tfsec can't express
  Enterprise policy governance
```

🔵 Short Interview Answer

> "Four main SAST tools for Terraform: `tflint` is a linter that catches syntax errors, invalid provider arguments (wrong instance type names), and naming convention violations before hitting the API. `tfsec` scans for security misconfigurations — open security groups, unencrypted S3 buckets, missing deletion protection. `checkov` is more comprehensive, covering compliance frameworks like CIS benchmarks and supporting multiple IaC formats. `terrascan` enables custom Rego policy rules for organization-specific requirements. In a CI/CD pipeline I run them in order: tflint for syntax, tfsec/checkov for security, and fail the PR on HIGH findings. Results are uploaded to GitHub's Security tab via SARIF format."

Topic 60: ⚠ State Encryption — At Rest and In Transit

🔵 What It Is (Simple Terms)

State encryption protects the sensitive data in Terraform state files — both when stored on disk or in S3 (at rest) and when transferred between Terraform and the backend (in transit). Given that state contains plaintext secrets, encryption is non-negotiable in production.

🔵 Encryption at Rest — S3 Backend

```
# Level 1: SSE-S3 (AWS-managed key) — baseline
terraform {
```

```
  backend "s3" {
    bucket  = "mycompany-terraform-state"
    key     = "prod/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true     # enables SSE-S3 automatically
    # AES-256, AWS manages the key
    # Anyone with S3 read permission can decrypt — no additional
protection
  }
}

# Level 2: SSE-KMS (Customer-managed key) — production standard
terraform {
  backend "s3" {
    bucket     = "mycompany-terraform-state"
    key        = "prod/terraform.tfstate"
    region     = "us-east-1"
    encrypt    = true
    kms_key_id = "arn:aws:kms:us-east-1:123456789012:key/abc123-def456"
    # Or use a key alias:
    # kms_key_id = "alias/terraform-state-key"

    # With SSE-KMS:
    # - S3 access alone is NOT enough to read state
    # - Must also have KMS Decrypt permission on this key
    # - All KMS API calls logged in CloudTrail
    # - Key policy can further restrict who can decrypt
  }
}
```

🔵 KMS Key Policy for State Encryption

```
# Dedicated KMS key for Terraform state
resource "aws_kms_key" "terraform_state" {
  description             = "KMS key for Terraform state encryption"
  deletion_window_in_days = 30
  enable_key_rotation     = true   # auto-rotate annually

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        # Key administrators — manage the key but can't use it
        Sid    = "KeyAdministration"
        Effect = "Allow"
        Principal = { AWS = "arn:aws:iam::123456789012:role/SecurityAdmin"
}
        Action = ["kms:Create*", "kms:Describe*", "kms:Enable*",
                  "kms:List*", "kms:Put*", "kms:Update*", "kms:Revoke*",
                  "kms:Disable*", "kms:Get*", "kms:Delete*",
```

```
"kms:ScheduleKeyDeletion"]
        Resource = "*"
      },
      {
        # Terraform execution roles — encrypt and decrypt state
        Sid    = "TerraformStateAccess"
        Effect = "Allow"
        Principal = {
          AWS = [
            "arn:aws:iam::123456789012:role/TerraformRole-Prod-
Networking",
            "arn:aws:iam::123456789012:role/TerraformRole-Prod-Compute",
          ]
        }
        Action   = ["kms:GenerateDataKey", "kms:Decrypt",
"kms:DescribeKey"]
        Resource = "*"
      },
      {
        # Deny all other principals
        Sid    = "DenyOthers"
        Effect = "Deny"
        Principal = { AWS = "*" }
        Action    = "kms:*"
        Resource  = "*"
        Condition = {
          StringNotLike = {
            "aws:PrincipalArn" = [
              "arn:aws:iam::123456789012:role/SecurityAdmin",
              "arn:aws:iam::123456789012:role/TerraformRole-Prod-*"
            ]
          }
        }
      }
    ]
  })
}

resource "aws_kms_alias" "terraform_state" {
  name          = "alias/terraform-state-prod"
  target_key_id = aws_kms_key.terraform_state.key_id
}
```

🔵 Encryption in Transit

```
# S3 bucket policy: deny non-HTTPS connections
resource "aws_s3_bucket_policy" "state" {
  bucket = aws_s3_bucket.terraform_state.id

  policy = jsonencode({
```

```
    Version = "2012-10-17"
    Statement = [
      {
        Sid       = "DenyNonHTTPS"
        Effect    = "Deny"
        Principal = "*"
        Action    = "s3:*"
        Resource  = [
          "${aws_s3_bucket.terraform_state.arn}",
          "${aws_s3_bucket.terraform_state.arn}/*"
        ]
        Condition = {
          Bool = { "aws:SecureTransport" = "false" }
        }
      }
    ]
  })
}

# This denies any API call that doesn't use TLS/HTTPS
# Prevents state from being read over unencrypted connections
```

🔵 Terraform Cloud — Encryption Included

```
Terraform Cloud state encryption:
  ✅ Encryption at rest: AES-256
  ✅ Encryption in transit: TLS 1.2+ for all API calls
  ✅ Customer-managed encryption keys (Terraform Enterprise)
  ✅ No configuration needed — always encrypted
  ✅ Access control via workspace permissions
  ✅ Audit log of all state access
```

🔵 Terraform Native State Encryption (Terraform 1.10+)

```
# Terraform 1.10 introduced native encryption for local and remote state

terraform {
  encryption {
    key_provider "pbkdf2" "my_passphrase" {
      passphrase = var.state_encryption_passphrase
    }

    method "aes_gcm" "my_method" {
      keys = key_provider.pbkdf2.my_passphrase
    }

    state {
```

```
      method = method.aes_gcm.my_method
      enforced = true   # refuse to read unencrypted state
    }

    plan {
      method = method.aes_gcm.my_method
    }
  }
}

# Also supports AWS KMS key provider:
terraform {
  encryption {
    key_provider "aws_kms" "prod_key" {
      kms_key_id = "arn:aws:kms:us-east-1:123456789012:key/abc123"
      region     = "us-east-1"
    }

    method "aes_gcm" "kms_method" {
      keys = key_provider.aws_kms.prod_key
    }

    state {
      method   = method.aes_gcm.kms_method
      enforced = true
    }
  }
}
```

🔵 Encryption Checklist

```
At-Rest Encryption:
  ✅ S3: encrypt = true + kms_key_id (customer-managed KMS key)
  ✅ S3: bucket default encryption configured
  ✅ KMS: key rotation enabled (annual automatic rotation)
  ✅ KMS: key policy restricts decrypt to Terraform roles only
  ✅ KMS: CloudTrail logging enabled for KMS API calls

In-Transit Encryption:
  ✅ S3 bucket policy: deny non-HTTPS (aws:SecureTransport = false)
  ✅ DynamoDB: uses HTTPS by default (no extra config needed)
  ✅ TF backend configuration uses HTTPS endpoints

Access Control (defense in depth):
  ✅ S3 bucket: block all public access (all 4 settings)
  ✅ S3 bucket: no ACLs (bucket owner enforced)
  ✅ IAM: separate roles per environment, least privilege
  ✅ S3: enable versioning for recovery
  ✅ S3: enable access logging for audit
```

🔵 Short Interview Answer

> "State encryption has two layers: at rest and in transit. For S3 at rest, use `encrypt = true` with a customer-managed KMS key (`kms_key_id`). SSE-KMS is better than SSE-S3 because the KMS key policy independently controls who can decrypt — even if someone gets S3 read permission, they also need KMS Decrypt on the specific key. All KMS API calls are logged in CloudTrail. For in transit, enforce HTTPS via an S3 bucket policy that denies connections where `aws:SecureTransport = false`. Terraform 1.10+ adds native encryption that encrypts state client-side before writing to the backend. Terraform Cloud handles all of this transparently with AES-256 at rest and TLS in transit."

Topic 61: Supply Chain Security — Provider/Module Verification

---

🔵 What It Is (Simple Terms)

Supply chain security in Terraform means ensuring that the **providers and modules you use are genuine, unmodified, and from trusted sources**. A compromised provider could exfiltrate credentials or create backdoors. A tampered module could create malicious resources.

🔵 Provider Supply Chain Security

The Lock File — Provider Integrity Verification

```
# .terraform.lock.hcl — generated by terraform init

provider "registry.terraform.io/hashicorp/aws" {
  version     = "5.31.0"
  constraints = "~> 5.0"

  hashes = [
    # h1: SHA-256 hash of the ZIP archive content (provider binary)
    "h1:abc123...",

    # zh: HashiCorp's signed hash (different algorithm)
    "zh:abc456...",

    # Platform-specific hashes
    "h1:xyz789...",   # linux_amd64
    "h1:def012...",   # darwin_arm64
  ]
}
```

```
# The lock file ensures:
# 1. Same provider version used by ALL team members
# 2. Same binary hash — provider hasn't been tampered with
# 3. If someone runs terraform init and gets a different hash, it FAILS

# terraform init verifies:
# a. Download the provider from registry
# b. Compute hash of downloaded archive
# c. Compare to hash in .terraform.lock.hcl
# d. If mismatch: ERROR — provider may have been tampered with

# Error you'd see:
# Error: Failed to install provider
#   The checksum for terraform-provider-aws_5.31.0_linux_amd64.zip does
not match
#   the pre-computed checksum stored in the dependency lock file.

# Always commit .terraform.lock.hcl to Git
git add .terraform.lock.hcl
git commit -m "chore: pin provider versions and hashes"
```

🔵 Multi-Platform Lock File

```
# By default, lock file only has hashes for the platform where you ran
init
# CI/CD on Linux but developers on Mac = different hashes needed

# Add hashes for all platforms you use:
terraform providers lock \
  -platform=linux_amd64 \
  -platform=linux_arm64 \
  -platform=darwin_amd64 \
  -platform=darwin_arm64 \
  -platform=windows_amd64

# This queries the registry and adds all platform hashes to the lock file
# Now terraform init works on any platform without re-downloading
```

🔵 Provider Verification Tiers

```
Tier 1: Official (HashiCorp maintained)
  Source: hashicorp/aws, hashicorp/google, hashicorp/azurerm
  Trust: Highest — HashiCorp develops and maintains
  Signed by HashiCorp's GPG key
  ✅ Use for major cloud providers
```

```
Tier 2: Partner (Vendor maintained, HashiCorp validated)
  Source: datadog/datadog, mongodb/mongodbatlas, pagerduty/pagerduty
  Trust: High — vendor-maintained, registry-verified
  ✅ Use for verified partner integrations

Tier 3: Community (Anyone can publish)
  Source: random_namespace/random_provider
  Trust: Low — no vetting process
  ⚠ Evaluate carefully: check source code, issue activity, stars
  ❌ Avoid for production if possible
```

🔵 Module Supply Chain Security

```
# Verified public modules — lower risk
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "= 5.1.4"   # exact pin — immutable version
  # ✅ Well-maintained, 1000s of stars, actively reviewed
}

# Git modules with commit SHA — maximum immutability
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=a1b2c3d4e5f6"
  # ✅ Commit SHA cannot be moved — truly immutable
  # Even if the tag is force-pushed, SHA stays the same
}

# Git modules with tag — good but tags can be force-pushed
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=v2.1.0"
  # ⚠ Tags can be force-pushed in theory (rare in practice)
  # Protect against this with tag protection rules in GitHub
}
```

🔵 Private Mirror for Air-Gapped Environments

```
# Organizations with strict network controls use provider mirrors
# to avoid downloading from the public registry

# Step 1: Mirror providers to local filesystem
terraform providers mirror /path/to/mirror \
  -platform=linux_amd64 \
  -platform=linux_arm64

# Creates:
```

```
# /path/to/mirror/registry.terraform.io/hashicorp/aws/5.31.0.json
# /path/to/mirror/registry.terraform.io/hashicorp/aws/terraform-provider-
aws_5.31.0_linux_amd64.zip

# Step 2: Configure Terraform to use the mirror
cat > ~/.terraformrc << 'EOF'
provider_installation {
  filesystem_mirror {
    path    = "/path/to/mirror"
    include = ["registry.terraform.io/*/*"]
  }
  direct {
    exclude = ["registry.terraform.io/*/*"]
  }
}
EOF

# Or network mirror (Artifactory, Nexus)
cat > ~/.terraformrc << 'EOF'
provider_installation {
  network_mirror {
    url     = "https://artifactory.mycompany.com/terraform-providers/"
    include = ["registry.terraform.io/hashicorp/*"]
  }
  direct {
    exclude = ["registry.terraform.io/hashicorp/*"]
  }
}
EOF
```

🔵 GPG Signature Verification

```
# HashiCorp signs all provider releases with their GPG key
# terraform init verifies signatures automatically

# Manually verify a provider:
# 1. Download provider and SHA256SUMS + SHA256SUMS.sig
curl -LO https://releases.hashicorp.com/terraform-provider-
aws/5.31.0/terraform-provider-aws_5.31.0_linux_amd64.zip
curl -LO https://releases.hashicorp.com/terraform-provider-
aws/5.31.0/terraform-provider-aws_5.31.0_SHA256SUMS
curl -LO https://releases.hashicorp.com/terraform-provider-
aws/5.31.0/terraform-provider-aws_5.31.0_SHA256SUMS.sig

# 2. Import HashiCorp's public key
gpg --recv-keys C874011F0AB405110D02105534365D9472D7468F

# 3. Verify signature
gpg --verify terraform-provider-aws_5.31.0_SHA256SUMS.sig terraform-
provider-aws_5.31.0_SHA256SUMS
```

```
# 4. Verify ZIP hash
sha256sum -c terraform-provider-aws_5.31.0_SHA256SUMS 2>/dev/null | grep
terraform-provider-aws_5.31.0_linux_amd64.zip
```

🔵 Dependency Review in CI/CD

```yaml
# GitHub Actions: review provider/module changes
name: Dependency Review

on:
  pull_request:
    paths:
      - '**.tf'
      - '.terraform.lock.hcl'

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Check for lock file changes — requires review
      - name: Check lock file
        run: |
          if git diff --name-only HEAD~1 | grep -q ".terraform.lock.hcl";
then
            echo "::warning::Lock file changed — provider version update
requires security review"
          fi

      # Verify no community providers without approval
      - name: Check provider sources
        run: |
          # Extract all provider sources from lock file
          grep "^provider" .terraform.lock.hcl | \
          grep -v "hashicorp\|aws\|google\|azure" && \
          echo "::error::Non-standard provider found — requires security
review" && exit 1 || true
```

🔵 Checkov Supply Chain Checks

```
# checkov has specific supply chain checks
checkov -d . --check CKV_TF_1   # Ensure Terraform module sources use
commit hash
checkov -d . --check CKV_TF_2   # Ensure Terraform Gitlab module registry
uses semver tag
```

```
# Example finding:
# Check: CKV_TF_1: "Ensure Terraform module sources use a commit hash"
#   FAILED for resource: module.vpc
#   File: /main.tf:1-5
#   Guidance: Use ?ref=<commit-sha> instead of ?ref=<branch-name>
```

🔵 Short Interview Answer

> "Supply chain security for Terraform focuses on two areas. For providers: the `.terraform.lock.hcl` file stores cryptographic hashes of provider binaries — if a provider is tampered with between releases, the hash mismatch causes `terraform init` to fail. Always commit the lock file to Git. For multi-platform teams, run `terraform providers lock -platform=linux_amd64 -platform=darwin_arm64` to include all platform hashes. For modules: pin to exact versions or commit SHAs — never floating branch references. In air-gapped environments, use `terraform providers mirror` to create a local mirror and configure `~/.terraformrc` to use it instead of the public registry. Finally, prefer HashiCorp Official and Partner tier providers over community providers for production workloads."

📊 Category 8 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight
55. Sensitive variables ⚠	sensitive=true = CLI redaction only, state = always plaintext	⭐⭐⭐⭐⭐
56. Secrets integration	manage_master_user_password, Vault dynamic, TF_VAR_ for injection	⭐⭐⭐⭐⭐
57. Least privilege IAM	One role per state file, iamlive for discovery, permission boundaries	⭐⭐⭐⭐⭐
58. OIDC auth	JWT → STS AssumeRoleWithWebIdentity, no static keys, restrict to branch	⭐⭐⭐⭐⭐
59. SAST tools	tflint (syntax), tfsec (security), checkov (compliance), terrascan (Rego)	⭐⭐⭐⭐
60. State encryption ⚠	SSE-KMS > SSE-S3, HTTPS-only bucket policy, TF 1.10 native encryption	⭐⭐⭐⭐⭐
61. Supply chain	Lock file hashes, commit SHA pins, private mirror, provider tiers	⭐⭐⭐⭐

🔑 Category 8 — Critical Rules

```
Secrets:
  sensitive = true → CLI redaction ONLY, state is still plaintext
  State file = security boundary, treat like a database of secrets
  Best: manage_master_user_password = no Terraform involvement
  Good: TF_VAR_ env vars injected by CI/CD — never in files
  Never: secrets hardcoded in .tf or committed in tfvars

IAM:
  One role per state file (per environment + component)
  Plan role = read-only, Apply role = read+write
  Trust policy: restrict to specific repo + branch + environment
  Permission boundary: caps permissions even if policy is misconfigured

OIDC:
  Requires id-token: write permission in GitHub workflow
  JWT → STS AssumeRoleWithWebIdentity → temporary credentials
  Credentials expire automatically — no rotation needed
  Restrict sub claim to specific repo:branch:environment

State Encryption:
  encrypt = true + kms_key_id (never SSE-S3 alone for production)
  S3 bucket policy: deny SecureTransport = false
  KMS key policy: restrict Decrypt to Terraform roles only
  Enable CloudTrail for S3 + KMS API audit logging

Supply Chain:
  Always commit .terraform.lock.hcl
  Run terraform providers lock for all platforms
  Pin modules to tags/SHA — never branches in production
  checkov CKV_TF_1: enforce commit hash for module sources
```

🎯 Category 8 — Top 5 Interview Questions to Master

1. **"Does marking a variable `sensitive = true` protect it from appearing in the state file?"** — No, sensitive only redacts CLI output, state is always plaintext JSON
2. **"How would you set up CI/CD for Terraform without storing any AWS credentials?"** — OIDC: OIDC provider in AWS, IAM role with trust policy restricting to repo/branch, `id-token: write` in workflow
3. **"What is the principle of least privilege for Terraform IAM roles?"** — One role per state file, minimum required permissions, plan vs apply role separation, permission boundaries
4. **"Walk me through how the Terraform lock file prevents supply chain attacks"** — lock file stores hashes, init verifies hash, mismatch = failure, always commit to Git, `providers lock -platform=` for multi-platform
5. **"What's the safest way to handle a database password in Terraform?"** — `manage_master_user_password = true` (AWS manages it entirely), or random_password → Secrets Manager, never as tfvar in committed files

---

> **Next:** Category 9 — Workflows, CI/CD & Terraform Cloud (Topics 62–72) Type `Category 9` to continue, `quiz me` to be tested on Category 8, or `deeper` on any specific topic.